

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 3

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Алгоритми сортування та їх складність. Порівняння алгоритмів сортування

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Порядок виконання роботи

1. Вивчити самостійно і записати (будь-яким способом) алгоритм бульбашкового сортування. Оцінити асимптотику алгоритму сортування методом бульбашки в найгіршому і в найкращому випадку. Порівняти за цими показниками бульбашковий алгоритм з алгоритмом сортування вставлянням. Чому на практиці бульбашковий алгоритм виявляється менш ефективним у порівнянні з сортуванням методом зливанням?

1.1. Алгоритм бульбашкового сортування

Код алгоритму на Python:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

1.2. Оцінка асимптотики

- Найгірший випадок: $O(n^2)$ (коли масив впорядкований у зворотному порядку).
- Найкращий випадок: $O(n)$ (якщо масив уже відсортований, завдяки перевірці змінного прапорця `swapped`).

1.3. Порівняння із сортуванням вставлянням

Обидва алгоритми мають однакову теоретичну складність у найкращому $O(n)$ та найгіршому $O(n^2)$ випадках. Проте сортування вставлянням зазвичай

працює швидше на практиці, оскільки виконує значно менше операцій перезапису пам'яті (елементи зсуваються, а не постійно міняються місцями через третю змінну).

1.4. Ефективність порівняно з сортуванням злиттям

На практиці бульбашкове сортування неефективне для великих масивів, тому що його складність $O(n^2)$ зростає квадратично. Сортування злиттям працює за принципом «розділяй і володарюй» і гарантує швидкість $O(n \log n)$ за будь-яких умов, що потребує значно менше порівнянь.

2. Оцінити асимптотичну складність алгоритму сортування зливанням, скориставшись основною теоремою рекурсії.

2.1. Оцінка складності сортування злиттям за основною теоремою рекурсії

Рекурентне співвідношення для Merge Sort має вигляд:

$$T(n) = 2T(n/2) + O(n)$$

Де:

- $a = 2$ (задача ділиться на дві підзадачі)
- $b = 2$ (розмір кожної підзадачі зменшується вдвічі)
- $f(n) = O(n)$ (час, необхідний на злиття підмасивів)

Обчислюємо значення $\log_b(a) = \log_2(2) = 1$. Оскільки функція $f(n) = \Theta(n^1) = \Theta(n^{\log_b(a)})$, ми маємо 2-й випадок основної теореми рекурсії. Відповідно, загальна складність алгоритму:

$$T(n) = \Theta(n \log n)$$

3. Вивчити і записати (будь-яким способом) самостійно алгоритм швидкого сортування. Оцінити асимптотичну складність алгоритму швидкого сортування, скориставшись основною теоремою рекурсії.

3.1. Алгоритм швидкого сортування

Код алгоритму на Python:

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    return quick_sort(left) + middle + quick_sort(right)
```

3.2. Оцінка складності за основною теоремою рекурсії

У середньому та найкращому випадку опорний елемент (pivot) ділить масив приблизно навпіл, що дає рекурентне співвідношення:

$$T(n) = 2T(n/2) + O(n)$$

Аналогічно до сортування злиттям, за 2-м випадком основної теореми рекурсії (де $a=2$, $b=2$, $f(n)=O(n)$), складність становить:

$$T(n) = O(n \log n)$$

Примітка: У найгіршому випадку (коли опорний елемент є найменшим або найбільшим у масиві) задача ділиться на 1 та $n-1$ елементів, рекурсія набуває вигляду $T(n) = T(n-1) + O(n)$, що дає складність $O(n^2)$. У цьому випадку класична основна теорема не застосовується.

Висновки: на цій практичній роботі я опанував основні алгоритми сортування та навчився методам аналізу їх асимптотичної складності.

Контрольні питання

1. Що таке асимптотична складність алгоритму сортування і чому вона важлива для порівняння алгоритмів?

- Асимптотична складність – це математична оцінка характеру зростання часу роботи (або об'єму пам'яті) алгоритму залежно від обсягу вхідних даних (n) при $n \rightarrow \infty$

- Вона важлива, бо дозволяє об'єктивно порівнювати ефективність різних алгоритмів між собою, не прив'язуючись до потужності конкретного комп'ютера, операційної системи чи мови програмування.

2. Які алгоритми сортування мають квадратичну складність у найгіршому випадку? Поясніть, чому це може бути проблемою для великих обсягів даних.

- Квадратичну складність ($O(n^2)$) мають: сортування бульбашкою (Bubble Sort), сортування вставлянням (Insertion Sort), сортування вибором (Selection Sort), а також швидке сортування (Quick Sort) у своєму найгіршому випадку.

- Це велика проблема для Big Data, адже час виконання зростає квадратично: якщо обсяг даних збільшити в 10 разів, алгоритму знадобиться вже в 100 разів більше операцій. На мільйонних масивах такі алгоритми просто зависнуть.

3. В чому полягає перевага сортування злиттям над сортуванням вставками для великих наборів даних?

- Сортування злиттям (Merge Sort) гарантує швидкість $O(n \log n)$ у будь-якому випадку (найкращому, середньому та найгіршому).
- Математично функція $n \log n$ зростає значно повільніше, ніж квадратична n^2 . Тому на великих невідсортованих наборах даних сортування злиттям виконується в тисячі разів швидше за сортування вставленням.

4. Які алгоритми сортування використовуються для сортування списків у стандартних бібліотеках мов програмування, таких як Python, Java або C++?

- Python: використовує Timsort (гібрид сортування злиттям та сортування вставленням).
- Java: використовує Timsort (для об'єктів/колекцій) та *Dual-Pivot Quicksort* (для масивів примітивних типів).
- C++: функція `std::sort` базується на Introsort (гібрид швидкого сортування, який за потреби перемикається на пірамідаліне сортування/Heap Sort, та сортування вставленням для малих ділянок).

5. Яка різниця між алгоритмами сортування злиттям і швидким сортуванням? У яких випадках краще використовувати кожен з цих алгоритмів?

- Сортування злиттям – стабільне, завжди працює за $O(n \log n)$, але вимагає багато додаткової пам'яті ($O(n)$) для створення копій масиву. Його краще брати для зв'язаних списків, зовнішнього сортування або коли критична стабільність елементів.
- Швидке сортування – нестабільне, у найгіршому випадку може сповільнюватися до $O(n^2)$, але працює "на місці" (In-place, пам'ять $O(\log n)$) і на практиці зазвичай випереджає сортування злиттям. Його краще обирати для стандартних масивів у RAM, коли ресурси пам'яті обмежені.

6. Які фактори слід враховувати при виборі алгоритму сортування для конкретної задачі?

- Розмір вхідного масиву (кількість елементів).
- Початковий стан даних (масив уже частково відсортований чи повністю хаотичний).
- Обмеження по оперативній пам'яті (чи є можливість виділяти додаткове місце).
- Необхідність збереження початкового порядку однакових елементів (стабільність сортування).
- Тип структури даних (динамічний масив, зв'язаний список тощо).